Project Executable Files

Date	04 July 2026
Team ID	
Project Name	Rising Waters – A Machine Learning Approach to Flood Prediction
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	No
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
Rising Waters/
|-- README.md Project overview and setup guide |--
requirements.txt Top-level dependencies (notebook + app)
|-- Dockerfile Container build for deployment
|-- Procfile Buildpack-style deployment (gunicorn)
|-- manifest.yml IBM Cloud Foundry deployment
manifest
|
|-- data/ | |-- rainfall in india 1901-2015.xlsx Historical
rainfall dataset | +-- flood dataset.xlsx Labelled weather/flood
dataset
|
|-- notebook/ | +-- Flood_Prediction_Analysis.ipynb EDA,
preprocessing, model training
|
|-- model/ | +-- floods.save Trained model + scaler
(joblib bundle)
|
+-- app/
|-- app.py Flask application
|-- requirements.txt App-level dependencies |--
features.json Feature list generated at startup
|-- templates/ | |-- base.html Shared layout
(navbar, footer)
| |-- home.html Home page
| |-- predict.html Prediction input page
| |-- flood_result.html Flood-likely result
page | +-- no_flood_result.html No-flood result
page
+-- static/css/style.css Stylesheet
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-gx15.onrender.com
Login Credentials (Demo)	Not applicable – the app has no authentication; all pages and the prediction tool are publicly accessible.
Platform / Hosting Provider	Render
Repository Link	https://github.com/lakshmilathish/rising-waters
Demo Video Link	https://drive.google.com/drive/folders/1y-EpVJrSjl9vZNK-FggB_ZLJStevhWzH?usp=drive_link

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

```
Pre-requisites: Python 3.11 or later, pip, and an internet connection (for
the Google Fonts / Font Awesome CDN links used by the UI).

Step 1: Clone the repository: git clone
https://github.com/lakshmilathish/rising-waters.git

Step 2: Navigate to the app folder:
cd "rising waters/app"

Step 3: Install dependencies:
pip install -r requirements.txt

Step 4: Start the application:
python app.py

Step 5: Open a browser at:
http://127.0.0.1:5000

(No database setup or .env configuration is required — the app loads the pre-
trained
model bundle model/floods.save directly.)
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	features.json is written to a relative path at startup, which can fail if the app is launched from a different working directory (e.g. via gunicorn).	Run the app from inside app/, or resolve the path with BASE_DIR (planned fix).
2	The hero/result banner images are loaded from external hosted URLs, so they won't display without internet access.	Planned: host the images locally under static/.
3	The model is trained on only 115 labelled records, so accuracy can vary with the train/test split and library versions.	Collect more labelled flood data to improve robustness in a future iteration.